🔎 What Is Your PC Connecting To? Spotting Hidden Connections and Remote Access Trojans

Fulllion Creative Works

Sources: Your computer is logging this! and How to know if your PC is hacked? Modern RATs, both by The PC Security Channel. Other sources are listed at the end.

The Short Version

- **What your PC looked up:** run `ipconfig /displaydns` to see recently resolved web addresses (Section 2).
- **What it's connected to right now, and which program:** use `netstat -ano`, PowerShell, Resource Monitor or TCPView (Section 3).
- **An address you don't recognize:** look it up on VirusTotal and with a whois search before drawing conclusions (Section 4).
- **What a RAT looks like:** a normal Windows program, such as PowerShell, running a scrambled command and talking to an unfamiliar server (Section 5).
- **Hardening:** Microsoft's attack surface reduction rules block the script tricks RATs rely on. PowerShell's execution policy doesn't (Section 7).

Contents

1. Why Network Activity Is the Best Clue
2. Check What Your PC Has Looked Up
3. See Live Connections and Which Program Owns Them
4. Look Up an Address You Don't Recognize
5. How a Modern RAT Hides
6. Signs Worth a Closer Look
7. Make Script Attacks Harder
8. If You Find Something
9. Sources

---

1. Why Network Activity Is the Best Clue

From the "Modern RATs" video.

A **remote access trojan (RAT)** gives an attacker control of your PC. Typical abilities include recording keystrokes, searching your files, using your microphone and camera, and sending what it collects back to the attacker. That last step is called **exfiltration**.

To do any of this, the RAT has to talk to the attacker's server, called a **command and control (C2)** server. Malware can disguise its files and processes, but it can't avoid making that connection. So the first thing to check is where your PC is connecting.

---

2. Check What Your PC Has Looked Up

From the "Your computer is logging this!" video, with added limits from Microsoft's documentation.

Before your PC connects to a website or server, it looks up the address in **DNS**. Windows keeps recent answers in a **DNS cache**, and you can read it.

2.1 Show the cache

Open the Start menu, type `cmd`, and open **Command Prompt**. Then run:

```
ipconfig /displaydns
```

The list is long. To save it to a file on your desktop instead:

```
ipconfig /displaydns > "%USERPROFILE%\Desktop\dns-cache.txt"
```

In PowerShell, this gives a shorter list with one line per name:

```
Get-DnsClientCache | Sort-Object Entry -Unique | Format-Table Entry, Data
```

2.2 What you'll see

In the video, the list was mostly Microsoft (Windows, Office, OneDrive, Teams and Windows Update), plus Google and Apple services. There were game launchers, and update checks from hardware makers such as Dell and NVIDIA. Some surprises were:

- **Analytics services,** such as one used by installed software to report usage.
- **Software the presenter hadn't opened in a long time,** still checking in on its own.
- **Updaters installed alongside hardware,** like a monitor's companion software.

That's the useful part: the list shows which software is quietly using the internet. Most of it is harmless, such as updates and sync, but it's a quick way to spot software you didn't know was installed. It can also catch an **infostealer**, a type of malware that sends your saved passwords to an attacker.

2.3 What the cache can't tell you

- **It's a snapshot.** Entries expire after a short time, so the cache only shows recent lookups.
- **It doesn't say which program made a lookup.** Section 3 covers that.
- **It includes your hosts file.** Microsoft notes the cache also holds entries loaded from the local `hosts` file, so not every line is a real lookup.
- **Browsers with secure DNS may skip it.** The video says your browser and private mode don't matter. That's true when the browser uses Windows' own lookup. But Chrome turns on **secure DNS** in

automatic mode by default, and it can then do encrypted lookups itself, with its own cache. Those may not appear here. VPNs can also change this.

2.4 Clearing it

The video shows that `ipconfig /flushdns` empties the cache. Microsoft describes it as a troubleshooting step. Only use it **after** you've looked, because it removes exactly what you're trying to check.

---

3. See Live Connections and Which Program Owns Them

The DNS cache shows names. These tools show **actual connections**, and the program behind each one.

3.1 Command Prompt: netstat

```
netstat -ano
```

Option	What it adds
`-a`	All connections, and ports your PC is listening on
`-n`	Numbers instead of names (faster)
`-o`	The **PID** (process ID) of the program that owns each connection

To find the program for a PID, open Task Manager (`Ctrl + Shift + Esc`), go to the **Details** tab, and look it up in the **PID** column.

3.2 PowerShell: connections with program names

This lists established connections with the program's name filled in:

```
Get-NetTCPConnection -State Established |
  Select-Object RemoteAddress, RemotePort,
    @{n='Process'; e={(Get-Process -Id $_.OwningProcess).ProcessName}} |
  Sort-Object Process | Format-Table -AutoSize
```

Change `Established` to `Listen` to see which programs are waiting for incoming connections.

3.3 Resource Monitor

Press `Windows Key + R`, type `resmon`, and press **Enter**. On the **Network** tab, **TCP Connections** lists each program, the remote address, and the port, and it updates live.

3.4 TCPView

TCPView is a free Microsoft tool from the **Sysinternals** suite, and it's the one the video uses.

1. Download TCPView from Microsoft and extract the zip file.
2. Open `tcpview64.exe`.

3. Each row shows the program, its local and remote addresses, and the connection's state. New connections are highlighted as they appear.

In the video, right-clicking a row gives options to copy the address and to run a **whois** lookup on it.

> **Seeing connections to `127.0.0.1` or `::1`?** Those stay on your own PC, between programs on it. They're normal, but local services can still be a security concern. See DNS Rebinding and the NVIDIA App's Local MCP Server.

---

4. Look Up an Address You Don't Recognize

From the "Modern RATs" video, with added notes.

1. **Copy the remote address,** an IP address or domain name, from TCPView or the list above.
2. **Search it on VirusTotal.** Paste it into the search box to see whether security vendors have flagged it.
3. **Check who owns it.** A whois search, from TCPView's right-click menu or any whois website, shows the organization behind a domain or IP range. As the video notes, it often won't give a clear answer.
4. **Search the domain name** in a search engine, as the video does with an analytics domain. That's often the quickest way to learn what a service is.

How to read the results:

- **Big cloud and content-delivery networks host thousands of sites on shared addresses.** An Amazon, Microsoft or Cloudflare address tells you little on its own.
- **A clean VirusTotal result doesn't prove an address is safe.** New attacker servers often aren't flagged yet.
- **Judge the program and the address together.** An unknown address used by your game launcher is far less worrying than one used by PowerShell.

---

5. How a Modern RAT Hides

From the "Modern RATs" video, with added Microsoft details.

The video runs a real RAT in a safe test environment. What it shows matters more than any single tool:

1. **No strange program names.** The process list showed only Windows' own tools: PowerShell, Command Prompt, and `wscript.exe` (Windows Script Host).
2. **A scrambled command.** PowerShell was running a long string of letters and numbers. It was **Base64**, a way of encoding text that hides it from a quick look.
3. **Decoding it showed the plan.** The command downloaded a JavaScript file into the public Downloads folder and started it, and that file was the RAT.
4. **The network gave it away.** The RAT connected to a C2 server, and the connection showed the attacker's IP address and port.

This is called **living off the land**: attackers use Windows' own tools so nothing looks out of place. As the video stresses, the program's name alone tells you nothing. The same system program can be doing normal work in one case and fetching malware in another.

5.1 See what a program is actually running

Task Manager can show each program's full command:

1. Open Task Manager and go to the **Details** tab.
2. Right-click a column header and choose **Select columns**.
3. Tick **Command line** and click **OK**.

A PowerShell process with `-EncodedCommand` (or `-enc`, `-e`) followed by a long string deserves a closer look. Microsoft documents that this option takes a Base64 string of **UTF-16LE** text.

5.2 Decode it safely

The video pastes the string into an online decoder. You can decode it in PowerShell instead, without running it:

```
$text = 'PASTE-THE-BASE64-HERE'
[Text.Encoding]::Unicode.GetString([Convert]::FromBase64String($text))
```

This only turns the string back into readable text. It never runs the command. **Don't paste the decoded command into PowerShell.**

---

6. Signs Worth a Closer Look

What you see	Why it matters	What to do
PowerShell, `wscript.exe` or `cmd.exe` connecting to the internet	Common tools for running malware scripts	Check its command line (5.1) and the address (Section 4)
A long Base64 string in a command line	Hiding what the command does	Decode it (5.2)
A program you don't recognize with an internet connection	It could be unwanted software, or malware	Search its name and file location
Programs running from `Downloads`, `Public`, `AppData\Local\Temp` or similar folders	Malware often runs from folders anyone can write to	Check the file with Windows Security or VirusTotal
Constant connections to one unfamiliar address	Could be a C2 server checking in	Look up the address (Section 4)
Something you don't recognize starting with Windows	Malware needs to survive restarts	Check it with Autoruns

---

7. Make Script Attacks Harder

The video's advice, corrected and expanded with Microsoft's documentation.

The video suggests disabling JavaScript, PowerShell scripts and Windows Script Host, since most people never use them. The idea is sound, but one common method doesn't work the way people think.

7.1 PowerShell's execution policy isn't a defense

Microsoft says plainly that the execution policy **isn't a security boundary**. It's a safety feature that stops you running scripts by accident. Malware gets around it easily, for example by passing commands with `-EncodedCommand` as in Section 5. Don't rely on it.

7.2 Attack surface reduction rules

These are part of **Microsoft Defender Antivirus**, and Microsoft says they work on any edition of Windows that includes it, including Home. Two of them block exactly what the video's RAT did:

Rule	GUID
Block execution of potentially obfuscated scripts	`5beb7efe-fd9a-4556-801d-275e5ffc04cc`
Block JavaScript or VBScript from launching downloaded executable content	`d3e037e1-3eb8-44c8-a917-57927947596d`

Turn them on from PowerShell, opened as administrator. Start in **audit mode**, which only logs what would have been blocked, so you can check nothing you need breaks:

```
$obfuscated = '5beb7efe-fd9a-4556-801d-275e5ffc04cc'
$jsvbs = 'd3e037e1-3eb8-44c8-a917-57927947596d'
Add-MpPreference -AttackSurfaceReductionRules_Ids $obfuscated, $jsvbs `
  -AttackSurfaceReductionRules_Actions AuditMode, AuditMode
```

When you're happy, run the same commands with `Enabled, Enabled` in place of `AuditMode, AuditMode`. To check which rules are set:

```
$p = Get-MpPreference
$p.AttackSurfaceReductionRules_Ids
$p.AttackSurfaceReductionRules_Actions
```

Notes from Microsoft:

- **Use `Add-MpPreference`, not `Set-MpPreference`.** `Set-MpPreference` replaces any rules you already have.
- **The obfuscated-scripts rule needs cloud-delivered protection** turned on in Windows Security.
- **Work PCs:** if your organization manages the PC, its settings override yours.

7.3 Smart App Control (Windows 11)

Smart App Control blocks apps it believes are malicious or untrusted, including unsigned ones. It's in the **Windows Security** app. Microsoft's FAQ says recent updates let you turn it back on without reinstalling Windows. But it also says a reset or reinstall may still be needed if you had optional diagnostic data turned off. There's no way to allow one blocked app as an exception.

---

8. If You Find Something

1. **Disconnect from the internet.** That cuts the attacker off and stops more data leaving.
2. **Don't wipe the PC yet,** and don't flush the DNS cache. Note down what you found.
3. **Follow the Windows Malware Removal Guide.**
4. **Change your important passwords from a different, clean device,** not the affected PC. That guide covers what else to do.

---

9. Sources

Videos (The PC Security Channel)

- Your computer is logging this!
- How to know if your PC is hacked? Modern RATs

Microsoft

- ipconfig
- Get-DnsClientCache
- netstat
- Get-NetTCPConnection
- TCPView
- About execution policies
- About pwsh (`-EncodedCommand`)
- Attack surface reduction rules reference
- Configure attack surface reduction rules
- Smart App Control FAQ

Other

- Google Chrome: Use secure DNS
- VirusTotal

Latest version: github.com/ImmuneMoon/PC-Network-Activity-and-RAT-Check

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.